

SASREC

训练时读取的是用户索引 ID，根据 ID 去偏移文件得偏移量，到行为序列文件中获取该用户历史信息

输入内容：用户行为序列（里面包括商品重要的静态特征和当时的动态特征值）、商品静态特征，商品多模态特征、用户文件偏移量、ID 映射索引

先看数据格式，举一个具体例子：

seq.jsonl 文件（用户行为序列）

```
// 每一行是一个用户的完整点击历史
// 格式: [(user_id, item_id, user_feat, item_feat, action_type, timestamp), ...]

[
  (101, 127, [{"性别": "男", "年龄": "20-30"}], [{"类目": "运动鞋", "品牌": "Nike"}, 1, 1700000001),
  (101, 128, [{"性别": "男", "年龄": "20-30"}], [{"类目": "篮球", "品牌": "Spalding"}, 1, 1700000002),
  (101, 129, [{"性别": "男", "年龄": "20-30"}], [{"类目": "运动袜", "品牌": "Nike"}, 1, 1700000005)
]
```

item_feat_dict.json 文件（商品特征）

```
{
  "127": {"类目ID": 5, "品牌ID": 12, "价格区间": 3},
  "128": {"类目ID": 8, "品牌ID": 7, "价格区间": 2},
  "129": {"类目ID": 5, "品牌ID": 12, "价格区间": 1}
}
```

creative_emb/ 目录（多模态向量）

```
creative_emb/
├── emb_81_32.pkl ← 商品标题的向量（32维）
│   内容: {"127": [0.1, 0.2, ..., -0.3], # 运动鞋标题向量
│         "128": [0.5, -0.1, ..., 0.4], # 篮球标题向量
│         ...}
│
├── emb_82_1024/ ← 商品图片的向量（1024维）
│   内容: {"127": [0.01, 0.02, ..., 0.99], # 运动鞋图片向量
│         ...}
│
```

2. seq_offsets.pkl — 文件偏移量目录（加载进内存）

```
# 存储每个用户数据在 seq.jsonl 文件中的字节起始位置
[
  0: 0,          # 用户reid=0 的数据从字节0开始
  1: 2048,       # 用户reid=1 的数据从字节2048开始
  2: 4096,       # 用户reid=2 的数据从字节4096开始
  3: 6144,       # 用户reid=3 的数据从字节6144开始
  ...
  999999: 2048000000 # 第100万用户
]
```

5. `indexer.pkl` — ID映射索引 (全加载进内存)

```
{
  'i': {
    # item索引: 原始ID → reid(重新编号的ID)
    'item_001': 1,
    'item_002': 2,
    'item_003': 3,
    ...
    'item_100000': 100000
  },
  'u': {
    # user索引: 原始ID → reid
    'user_001': 0,
    'user_002': 1,
    'user_003': 2,
    ...
  },
  'f': {
    # feature索引: 特征值 → reid
    '100': ['品类A': 1, '品类B': 2, '品类C': 3, ...],
    '117': ['品牌X': 1, '品牌Y': 2, '品牌Z': 3, ...],
    ...
  }
}
```

物品的静态特征和多模态特征如何全部变为 32 维向量？用户向量用同样的方式变为 32 维离散特征：取值有限，可比较，维护一个随机初始化的 `embedding` 表格，直接查表。对于所有的查表后的离散特征，进行 `concat`。

变长列表 `array` 特征：如小明的兴趣标签[1,2,3,4]和小妹的兴趣标签[2,6]。创建 `embedding` 表，维护兴趣标签内的内容。将小妹和小明的兴趣标签用 `padding = 0` 变为相同长度，查表，将多个 32 维向量求和，将小明的四个兴趣变为一个综合兴趣

Embedding 特征：文字特征原始为 1024 维，图片特征为 4096 维，直接线性变换即降维连续数值 `continual`（iPhone 价格=9999.0），实数非整数 ID：

取值无限，无法查表，不用变为 32 维，而是该物品有几个连续特征就是几维。方法是先将每个连续特征进行归一化，随后将所有归一化后的连续特征 `concat`。

最后拼接所有向量，变为更高维度，再走全连接层（DNN）变为 32 维。此时一个物品得到了 32 维的向量，可以和用户历史序列向量进行点积进行匹配、找相似物品、送到后续步骤 RQ-VAE 生成 Semantic ID。此时用户历史序列的形状为，每个物品变为了 32 维向量的形式。拼接顺序为：物品 IDembedding、Sparse 特征、array、continual、emb 特征。

此时用户的兴趣标签为一串商品，每个商品可以表示为一个 32 维的向量。

将用户信息和历史商品一起送往 `transformer`，保留 100 个历史记录，不够就 `padding = 0`。经过 `transformer` 后每个位置有 32 维向量，我们有一个标记位来维护哪些位置是商品而非 `padding` 和用户。目标是使得用户表征（预测物品前一个位置的 32 维向量）和正样本的点积大，负样本点积小。

用的 BCE loss，公式为 $L = -[y \times \log(p) + (1-y) \times \log(1-p)]$ ， y = 真实标签（0 或 1）、 p = 预测概率（0 到 1 之间）、 $\log$ = 自然对数。具体到我的实验中，若正样本点积为 0.8，则预

测为正的的概率为 $\text{sigmoid}(0.8)$, loss 的左半边为 $-\log(\text{sigmoid}(0.8))$, 和负样本的右半边相加即可。

对于一个用户序列, Loss 为所有位置的正样本 loss 平均+负样本 Loss 平均

为什么用 BCE loss: 推荐本质是二分类, 用户会点击这个物品吗? 是/否。正样本为会点击 (标签=1), 负样本为不会点击 (标签=0)。BCE Loss 专门用于二分类。

为什么 $\text{sigmoid}(\text{点积}) = \text{概率}$: 因为 sigmoid 当 $x=0$ 时值为 0.5, 点积越大, sigmoid 越大, 值域为 $[0,1]$

可学习的绝对位置编码 (为什么不用 RoPE? 推荐场景序列长度固定 ($\text{maxlen}=101$), 绝对位置足够; RoPE 的相对位置优势在超长序列场景更明显。) **decoder-only 架构**, 只走了 **mask-self-attn**, **2 层 decoder** (因为输入序列的长度为 101, 即只保留用户历史 100 个购买记录)。

为什么不用余弦位置编码: 没有参数不可训练。NLP 里位置规律比较通用。例如词序结构是语言固有规律。而推荐系统里, 不同数据集时间模式差异极大。例如电商最近行为极重要; 音乐长期偏好更重要; 短视频超短期兴趣最重要。因此让 **position embedding** 自己学习更灵活。

真实推理时, 我们不需要走 transformer (只在训练时, 用来训练 **feat2emb** 的参数), 因为经过上一步, **feat2emb** 已经被训练好了, 由 **feat2emb** 给每个历史物品一个 32 维的向量表示, 供后面的模块使用

模型的数据流可以概括为:

多种异构特征 → 各自 Embedding/变换 → 拼接 → DNN 压缩到 **hidden_units** 维 → 加位置编码 → Transformer 编码器 → 输出序列表征

效果怎么样? 冷启动商品的 Recall@50 从纯 ID 方案的 3.2% 提升到 18.7%, 接近 5 倍提升。整体商品表征质量显著提高, 语义相似的商品在向量空间中距离更近。

它怎么解决的? 构建了一个增强版 SASRec 模型。原版 SASRec 只处理 itemID 序列, 我们的版本把文本 Embedding (1024 维)、图片 Embedding (3584 维)、类目标签、属性特征等多模态信息全部融合进去。不同维度的特征通过线性变换统一映射到 32 维, 拼接后过 DNN 压缩。用 Transformer 编码器做序列建模提供训练信号, 训练完成后只用特征融合层导出商品向量。

它解决什么问题? 传统 ID Embedding 对冷启动商品无能为力, 因为没有交互数据就训不出有意义的向量。我们需要一种不依赖交互数据、靠商品自身信息就能生成高质量向量的方法。

优点: 用户行为序列延迟加载, 可以在用户持续增多后大幅减少内存占用 (因为不用存进去), 同时也可以通过调整启动参数来让图片、标题向量不进入内存

初始化策略：所有参数用Xavier 初始化，这是 Transformer 类模型的常规操作。但有一个细节，所有 EmbeddingTable 的第 0 行（padding 位置）被手动置零。这保证了 padding 位置的 Embedding 始终是零向量，不会对模型产生干扰。

Xavier initialization 本质上也是随机初始化。它和“普通随机初始化”的区别，不在于“是不是随机”，而在于随机数的方差（scale）是被精心设计过的，核心思想是让每层输入和输出的方差尽量保持一致

SASREC 训练开始时，item embedding、position embedding 以及 attention 里的 WQ,WK,WV 等参数本质上都是随机初始化的，只不过通常不会“随便随机”，而是采用 Xavier initialization 或小方差 Normal initialization, 让参数初始尺度保持稳定，避免 attention 的 QKT 在初期过大导致 softmax 塌缩、或过小导致所有位置注意力接近平均，从而保证深层 Transformer 中激活值和梯度能够稳定传播，后续再通过 next-item prediction 的 loss 反向传播逐步学出 item 语义、时间规律和用户兴趣模式。

训练-验证-导出的节奏：每个 epoch 结束后做三件事，在验证集上计算 loss，监控是否过拟合，保存模型 checkpoint（文件名里带 global_step 和 valid_loss，方便回溯），导出全量 itemembedding。每个 epoch 都导出一次 embedding，这样即使后续 epoch 过拟合了，也能回退到之前的版本。

L2 正则化的位置：代码里有一个容易被忽略的细节：L2 正则化只加在 item_emb 上，而不是所有参数。这是因为 itemEmbedding 是最容易过拟合的部分（热门item 被频繁更新，冷门item几乎不动），对它做正则化可以有效缓解这种不均衡。

这套设计带来了什么提升：

解决冷启动：传统 IDEmbedding 对新商品无能为力。但在这套方案里，即使一个商品从未被任何用户点击过，只要它有文本描述、有图片、有类目标签，feat2emb 就能为它生成一个有意义的向量。这个向量天然就和内容相似的老商品靠得很近，后续的 RQ-VAE 也能给它分配合理的语义 ID。

表征质量更高：单一模态的信息是片面的，文本描述可能很笼统，图片可能有歧义，类目标签可能太粗。但多模态融合后，不同信息源互相补充、互相校验，最终的向量能更准确地刻画商品的本质特征。

为下游任务提供统一接口：不管原始特征有多少种、维度差异有多大，经过 sasrec 这一步，所有商品都被统一表示为 hidden_units 维的向量。下游的 RQ-VAE 不需要关心特征是怎么来的，只需要处理一个干净的向量矩阵。这种“关注点分离”的设计让整条流水线的每个环节都可以独立迭代优化。

序列训练信号的加持：虽然最终导出时只用了特征融合层，但训练阶段的 Transformer 编码器提供了关键的监督信号：它迫使特征融合层学会产出“对序列预测有用”的向量。换句话说，Transformer 是一个“教练”，它通过 nextitemprediction 任务，倒逼融合层把真正重要的信息编码进向量里，而不是简单地把所有特征堆在一起。

sasrec 文件夹的本质是一个“多模态特征融合+序列训练”的组合拳。它不是为了做一个多好的序列推荐模型，而是借助序列推荐的训练范式，为每个商品锻造出一条高质量的语义向量。这条向量是整个生成式推荐系统的地基——地基打得越扎实，上面的 RQ-VAE 和大模型才能发挥得越好。

Self-Attn 解决的问题：

SASREC 不认为用户兴趣是静态的。传统 Matrix Factorization 会认为用户有一个固定兴趣向量，但 SASREC 认为用户兴趣是动态变化的。

比如前面买篮球鞋 → 说明用户可能在运动；后面突然看 PS5 → 兴趣可能切换到游戏

所以用户最近行为比远古行为更重要，但远古行为有时也可能突然变重要。用户兴趣不是固定向量，而是一个动态上下文相关状态

这就是 Self-Attention 要解决的问题。

六、完整配置表格	
用户数里	约50万
商品数里	约100万
训练数据里	约1500万行为记录
商品信息存储	约5-6GB内存
batch_size	128
maxlen	101
hidden_units	32
num_blocks	1
num_epochs	3
learning_rate	0.001
显存占用	约2GB
卡配置	1张4090就够了
训练时间	约1小时

FFN 中用 RELU